



I'm trying to understand how GPU program execution works internally, and several concepts are still unclear to me.

I know that:

A GPU's Streaming Multiprocessor (SM) is split into multiple partitions

Each partition is further divided into smaller units (possibly "cores" or similar components) of different types (e.g., flow sensors, fuses, etc.)

User programs must be translated into blocks that get assigned hierarchically: first to the GPU/SM, then to SM partitions, and finally to cores within those partitions

Various scheduling systems exist (e.g., thread manager, warp scheduler, 32 threads per warp)

There are hierarchical concepts like blocks, block threads, and individual threads that are confusing me

Could you create a detailed markdown document that:

Clearly explains how GPU program execution works with a simple, concrete example

Covers all four hierarchical components of the GPU: SM, partitions, cores, and threads

Includes ASCII diagrams or visual representations where needed to clarify the concepts

Uses simple, accessible language for a new engineer who wants to understand GPU architecture deeply

The goal is to make these hazy concepts crystal clear for someone new to GPU internals.

This version is organized, specific, and clearly communicates what you need while maintaining all your original requirements.

Here's a beginner-friendly markdown "mini-guide" to how a GPU runs your program, from threads all the way down to SM partitions and cores.

1. Big picture: how a GPU runs work

At a high level, a GPU is a chip with many identical processing blocks, called **Streaming Multiprocessors (SMs)** on NVIDIA GPUs. Each SM is like a small parallel CPU cluster that can run thousands of lightweight threads. ^{[1] [2] [3]}

When you launch a GPU kernel (a function to run on the GPU), you do not manually place threads on SMs or partitions. Instead:

- You tell the system how many **blocks** and how many **threads per block** you want.
- The GPU runtime and hardware:
 - Distribute **blocks** across SMs.
 - Inside each SM, split threads into **warps** of 32 threads.

- Each warp is assigned to one **SM partition**.
- The partition's **warp scheduler** issues instructions to the underlying **cores** (CUDA cores, tensor cores, etc.).^{[4] [5] [1]}

So the conceptual hierarchy from your code to hardware is:

- Your code: **Grid** → **Blocks** → **Threads**
- Hardware: **GPU** → **SMs** → **SM partitions** → **Cores**, executing **warps**.

2. Programming view: grid, blocks, threads, warps

From CUDA (and similar models) you see a software hierarchy.^{[6] [1] [7]}

2.1 Main concepts

- **Thread**: The smallest unit of execution in your kernel; runs your kernel function on one element (e.g., one pixel or one array element).^{[5] [1]}
- **Block (thread block)**: A group of threads that:
 - Run on the same SM.
 - Can share fast on-chip **shared memory** and can synchronize with `__syncthreads()`.^{[6] [7]}
- **Grid**: All the blocks launched for one kernel call.^{[1] [7]}
- **Warp**: Internally, the GPU groups 32 consecutive threads of a block into a **warp**.^{[8] [7] [5]}
The warp is the real hardware execution unit: all threads in a warp execute the same instruction in lockstep (SIMT).^{[5] [1]}

2.2 Simple ASCII picture

A single kernel launch:

```
Grid
├─ Block 0
│  ├── Threads 0..31  -> Warp 0
│  ├── Threads 32..63 -> Warp 1
│  └── ...
├─ Block 1
│  ├── Threads 0..31  -> Warp 0
│  └── ...
└─ Block N-1
```

Each block is mapped entirely to one SM, but one SM can host many blocks at once (depending on resource limits).^{[1] [2]}

3. Hardware view: GPU → SM → partition → cores

Now let's map those software concepts to physical hardware resources.

3.1 GPU and SMs

- A GPU chip has many SMs (NVIDIA) or CUs (AMD).^[3]
- Each SM:
 - Has its own register file, shared memory, warp schedulers, and a pool of execution units (CUDA cores, tensor cores, special function units).^{[1] [3]}
 - Receives some number of blocks from the global work queue.^{[5] [2]}

You can think of each SM as a “mini-GPU factory” that:

- Stores many warps (hundreds or more) ready to run.
- Chooses some warps each cycle to execute.^{[2] [5]}

3.2 SM partitions (sub-partitions)

Modern SMs (e.g., NVIDIA Ampere, Hopper, H100) are internally split into **several partitions**, sometimes called sub-partitions.^{[9] [10] [4]}

Typical behavior:

- Each SM has multiple **warp schedulers**, often one per partition.^[4]
- Warps assigned to that SM are distributed across these partitions.^{[10] [9]}
- Each partition has its own:
 - Warp scheduler.
 - Dispatch units.
 - Slice of execution units (e.g., CUDA cores and related pipelines).^{[1] [4]}

You do not program partitions explicitly; they are micro-architectural details the hardware uses to increase parallelism.

3.3 Cores and special units

Inside each SM partition you have different types of **cores**:

- **CUDA cores** (or stream processors): basic integer and floating-point ALUs; they execute most arithmetic instructions.^{[1] [3]}
- **Tensor cores** (on recent NVIDIA GPUs): specialized matrix-multiply units for ML workloads.^[1]
- **Special Function Units (SFUs)**: handle transcendental operations like sine, cosine, reciprocal, etc.^[1]
- Load/store units, texture units, etc.^[1]

These are the physical units that actually execute the individual instructions of your kernel.

3.4 ASCII “zoom-in” diagram

From top to bottom:

```
[ GPU ]
├─ SM 0
│   ├── Partition 0
│   │   ├── Warp scheduler
│   │   ├── CUDA cores
│   │   ├── Tensor cores
│   │   └── SFUs
│   ├── Partition 1
│   ├── Partition 2
│   └── Partition 3
├─ SM 1
└─ SM M-1
```

A warp “lives” on one SM and is assigned to exactly one partition within that SM. ^[9] ^[10] ^[4]

This diagram from an SM description shows partitions with their own warp schedulers, cores, and pipelines, which matches the ASCII view. ^[4] ^[1]

4. From kernel launch to cores: step-by-step example

Let’s walk through a concrete, simplified example: vector addition.

4.1 Example kernel and launch

Pseudo-CUDA:

```
__global__ void add_vectors(float* a, float* b, float* c, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        c[idx] = a[idx] + b[idx];
    }
}
```

Launch parameters:

```
int N = 1'000'000;
int threadsPerBlock = 256;
int numBlocks = (N + threadsPerBlock - 1) / threadsPerBlock;
// So numBlocks ≈ 3907

add_vectors<<<numBlocks, threadsPerBlock>>>(a, b, c, N);
```

This defines:

- Grid: ~3907 blocks.

- Each block: 256 threads.
- Within a block: $256 / 32 = 8$ warps. ^[5] ^[7]

4.2 Mapping blocks to SMs

1. The driver enqueues this kernel launch to the GPU. ^[5] ^[2]
2. The GPU hardware maintains a **global pool of blocks** for this kernel.
3. Each **SM** requests blocks when it has room:
 - Block 0 → SM 3
 - Block 1 → SM 7
 - Block 2 → SM 3
 - ...
4. A block stays on one SM for its whole lifetime. ^[1] ^[7] ^[5]

You do not control which block goes to which SM; that's managed by hardware.

4.3 Mapping threads to warps inside an SM

Inside a block (256 threads):

- Threads 0–31 → Warp 0
- Threads 32–63 → Warp 1
- ...
- Threads 224–255 → Warp 7 ^[5] ^[7]

Each warp is a group of 32 threads that executes one common instruction at a time (SIMT). ^[1] ^[5]

ASCII:

```
Block on SM 3
├─ Warp 0: threads 0..31
├─ Warp 1: threads 32..63
├─ Warp 2: threads 64..95
├─ Warp 3: threads 96..127
├─ Warp 4: threads 128..159
├─ Warp 5: threads 160..191
├─ Warp 6: threads 192..223
└─ Warp 7: threads 224..255
```

These warps are registered with one of the SM's partitions.

4.4 Mapping warps to SM partitions

Suppose the SM has 4 partitions and 64 resident warps in total. A typical mapping might be: ^[9]
^[10] ^[4]

- Partition 0 handles 16 warps.
- Partition 1 handles 16 warps.
- Partition 2 handles 16 warps.
- Partition 3 handles 16 warps.

Each partition's warp scheduler repeatedly:

1. Picks a ready warp from its assigned warps (one that is not waiting on memory or sync). ^[4]
^[5]
2. Issues the next instruction of that warp to cores in its partition. ^[1] ^[4]

You do not see or control this mapping; it's purely hardware scheduling.

4.5 Execution on cores

Consider one instruction from your kernel: `c[idx] = a[idx] + b[idx];`

At the hardware level for a single warp:

1. The warp's instruction is decoded.
2. 32 threads in the warp each:
 - Load `a[idx]` and `b[idx]` from memory (memory stage).
 - Perform a floating-point add on CUDA cores. ^[1] ^[3]
 - Store `c[idx]` back to memory.
3. Different warps may be issuing different instructions at the same time on different partitions, as long as there are enough cores and pipelines available. ^[4] ^[1]

The warp scheduler interleaves warps to hide memory latency: when one warp stalls on memory, another warp runs. ^[5] ^[2]

5. Tying together the four levels you asked about

Let's explicitly connect your four hierarchical components with the programming model.

5.1 GPU SM

- A **Streaming Multiprocessor (SM)** is the main execution block of the GPU:
 - Hosts many blocks and warps.
 - Owns resources: registers, shared memory, warp schedulers, cores. ^[1] ^[3]
- A block executes entirely on one SM. ^[7] ^[1]

5.2 SM partition

- An **SM partition** is a slice of an SM containing:
 - One warp scheduler.
 - A subset of the cores and pipelines.
- Warps assigned to the SM are distributed among partitions, and each partition independently schedules its warps. [\[9\]](#) [\[10\]](#) [\[4\]](#)

You do not specify or see partitions from CUDA code; they are internal.

5.3 Cores (CUDA cores, tensor cores, etc.)

- These are the **physical execution units**:
 - CUDA cores: integer and FP operations. [\[1\]](#) [\[3\]](#)
 - Tensor cores: matrix multiply-accumulate units.
 - SFUs: special math functions. [\[1\]](#)
- Warp instructions are broken down into operations on these cores.

5.4 Threads and warps

- Your code defines **threads** inside **blocks**.
- Hardware groups threads into **warps of 32**. [\[8\]](#) [\[5\]](#) [\[7\]](#)
- The warp is the scheduling and execution unit for partitions and cores.

5.5 Summary table

Concept from your question	Programming term	Hardware entity / detail
Entire GPU	GPU device	Many SMs on one chip [2] [3]
“GPUSM”	Thread block executes here	Streaming Multiprocessor (SM) [1] [3]
SM split into partitions	Not visible in code	SM sub-partitions, each with a warp scheduler [9] [10] [4]
“Cores in a partition”	CUDA cores, tensor cores	Execution units in each partition [1] [3]
Blocks	<code><<<numBlocks, blockDim>>></code>	Units scheduled to an SM [5] [1]
Threads	<code>threadIdx, blockIdx</code>	Grouped into warps of 32 [8] [5] [7]
Warp scheduler / 32 threads	Warp	Selects which warp runs on cores each cycle [8] [4] [5]

6. Mental model you can keep in your head

For a new engineer, a simple mental picture:

1. You describe work in terms of:

- Many blocks, each with many threads.

2. The GPU:

- Spreads blocks over its SMs.
- Inside each SM, splits threads into warps of 32.
- Assigns warps to internal SM partitions.
- Each partition's warp scheduler feeds its pool of cores with instructions from ready warps.

If you remember: “I write grids/blocks/threads, but the hardware runs SMs/partitions/warps/cores,” you’ll be able to reason about performance and behavior without needing every micro-architectural detail. ^[5] ^[1] ^[2]

If you’d like, next step I can add:

- A second markdown document just focusing on “performance tuning knobs” (block size, occupancy, divergence) tied back to this hierarchy.

✱

1. <https://www.abhik.ai/concepts/gpu-computing/shared-multiprocessor>

2. https://ajdillhoff.github.io/notes/cuda_architecture/

3. <https://docs.modular.com/glossary/gpu/streaming-multiprocessor/>

4. <https://modal.com/gpu-glossary/device-hardware/warp-scheduler>

5. <https://doc.sling.si/en/workshops/programming-gpu-cuda/02-GPU/01-exemodel/>

6. [https://en.wikipedia.org/wiki/Thread_block_\(CUDA_programming\)](https://en.wikipedia.org/wiki/Thread_block_(CUDA_programming))

7. <https://developer.codeplay.com/products/computecpp/ce/2.6.0/guides/sycl-for-cuda-developers/execution-model.html>

8. <https://forums.developer.nvidia.com/t/question-about-warp-execution-and-the-warp-scheduler/336310>

9. <https://stackoverflow.com/questions/79261161/how-do-warps-map-onto-sm-sub-partitions-in-a-gpu>

10. <https://forums.developer.nvidia.com/t/how-to-program-different-behaviors-of-4-partitions-in-1-sm-in-ada-arch/30917>